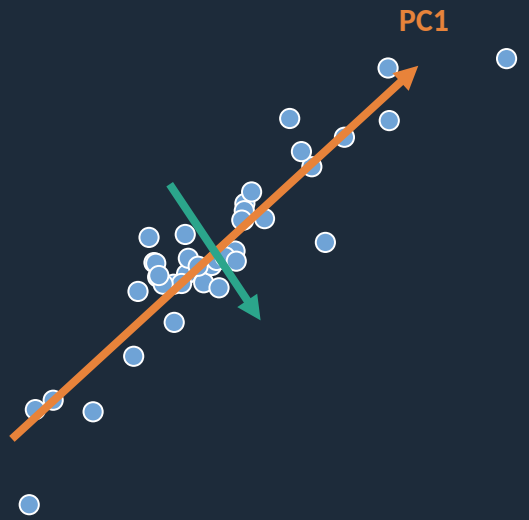
Dimensionality Reduction

PCA (Principal Component Analysis)



What Is Dimensionality Reduction?

Dimensionality reduction is the process of reducing the number of input features (dimensions) in a dataset, while keeping as much of the important information as possible.

Original data

n features (x_1, x_2, …, x_n)



Reduced data

k features (z_1, …, z_k), k < n

Same rows (examples), fewer columns (features).

Examples

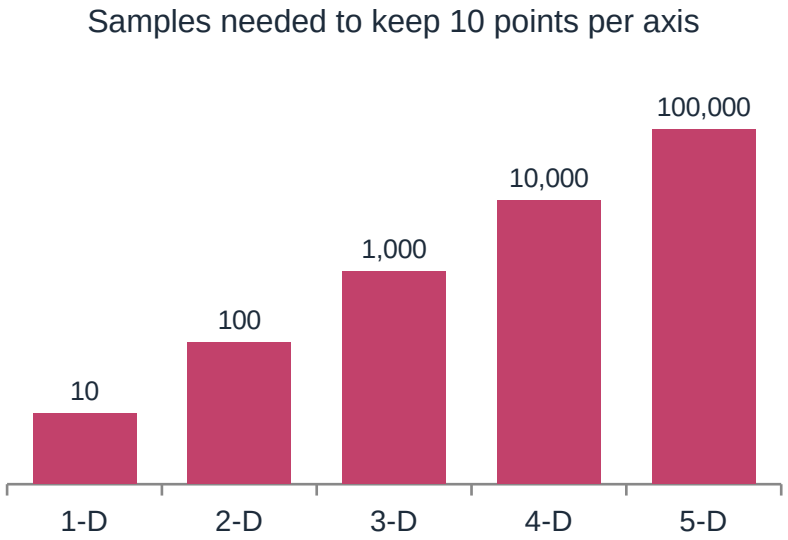
Images: 64 × 64 photo = 4,096 pixel features → about 50 components

Surveys: 30 questions → 3 underlying factors (e.g. price, quality, service)

Visualisation: A 100-feature dataset → 2 features we can plot

Why We Need It: The Curse of Dimensionality

Curse of dimensionality (term coined by Richard Bellman): As the number of features grows, the space grows exponentially, so the data becomes very sparse and many algorithms work badly.



Each extra feature multiplies the data needed by 10 (log scale).

Sparse data: Points end up far apart; there is too little data to cover the space.

Distances lose meaning: All points look almost equally far, which hurts KNN, K-means and clustering.

Overfitting: Models learn noise from features that do not matter.

Slow and costly: More features = more computation and memory.

Hard to visualise: We can only plot 2 or 3 dimensions.

Types of Dimensionality Reduction

Dimensionality reduction beats the curse of dimensionality in two ways:

Feature selection

Keep a subset of the original features and drop the rest.

e.g. keep “age” and “income”, drop “customer ID”

Methods: Filter, wrapper and embedded methods

Feature extraction

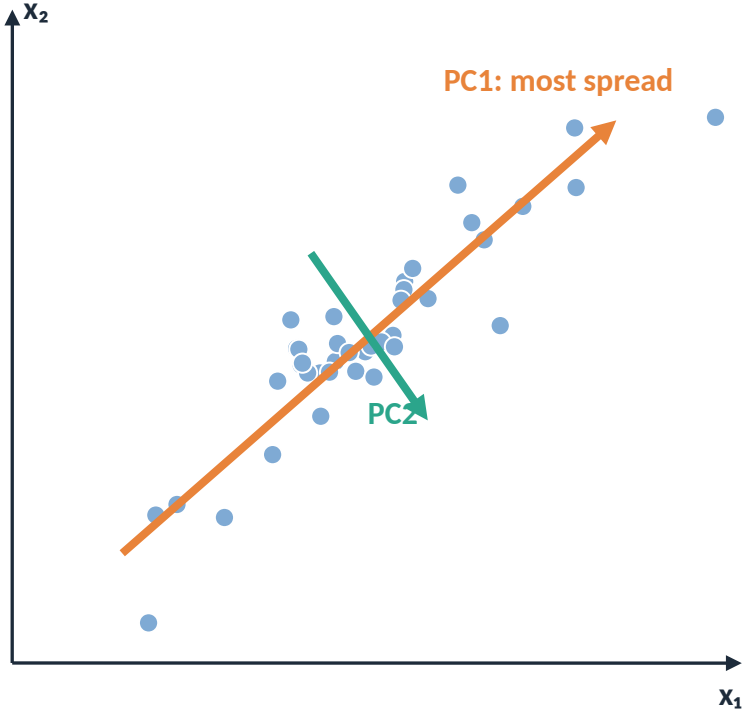
Create a few new features by combining the original ones.

e.g. PC1 = 0.56 × height − 0.83 × weight

Methods: PCA, LDA, t-SNE, autoencoders

What Is PCA?

- PCA is an unsupervised technique that finds new axes, called principal components, along which the data varies the most.
- PC1 points in the direction of maximum variance (spread). PC2 is perpendicular to PC1 and captures the next most variance, and so on.
- The components are uncorrelated with each other.
- Keep only the first few components → fewer dimensions, most of the information.



Correlated 2-D data: almost all the spread lies along PC1.

Where and Why PCA Is Used

PCA is an unsupervised learning technique: it uses only the input features X and never looks at labels. It finds structure (directions of variance) in the data itself.

Before clustering

K-means and hierarchical clustering rely on distances. In fewer, uncorrelated dimensions, distances are more meaningful and clustering runs faster.

Visualisation

Project 10, 100 or 1000 features onto PC1 and PC2 to plot the data and spot clusters or outliers by eye.

Pre-processing for supervised models

Fewer inputs for regression or classification → faster training, less overfitting, no multicollinearity.

Compression & noise removal

Keep the top components of images or signals; small components are mostly noise (e.g. eigenfaces).

Exploratory data analysis

See which original features drive most of the variation (the weights in each eigenvector).

Why it works

Real features are often correlated, so a few directions hold most of the information.

Key Terms Before We Start

Mean
The average of a feature. Used to centre the data.

Variance
How spread out one feature is around its mean.

Covariance
How two features change together: + rise together, − one rises as the other falls, 0 unrelated.

Covariance matrix (S)
A square table of all variances (diagonal) and covariances (off-diagonal).

Eigenvector
A direction that S only stretches, never turns. In PCA it is a principal component direction.

Eigenvalue (λ)
How much S stretches that direction = the variance of the data along it.

Core relationship: an eigenvector u and eigenvalue λ of S satisfy $S\mathbf{u} = \lambda\mathbf{u}$

PCA: The Steps

1

Mean

Compute the mean of each feature (and centre the data).

2

Covariance matrix

Compute the variances and covariances of the features.

3

Eigenvalues

Solve $\det(S - \lambda I) = 0$.

4

Eigenvectors

Solve $(S - \lambda I)U = 0$, then make each a unit vector.

5

Principal components

Keep the top eigenvector(s) and project: $e_1^T(X - \bar{X})$.

6

Geometric meaning

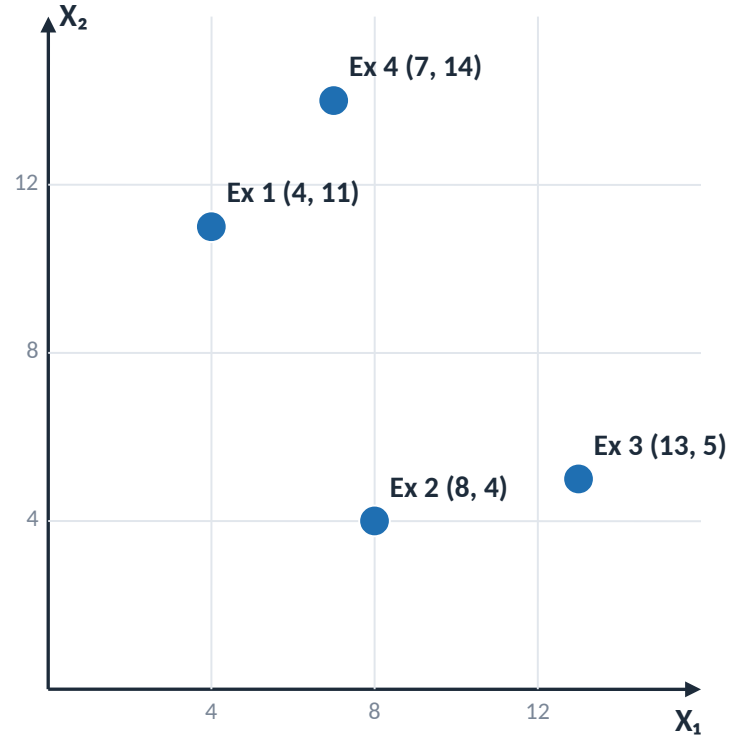
Interpret the new axes e_1, e_2 and the projected points.

Solved Example: The Problem

Given the data in the table, reduce the dimension from 2 to 1 using the Principal Component Analysis (PCA) algorithm.

Feature	Example 1	Example 2	Example 3	Example 4
X_1	4	8	13	7
X_2	11	4	5	14

- 2 features (X_1, X_2) and N = 4 examples.
- Each example is a point (X_1, X_2) in 2-D space.
- Goal: replace (X_1, X_2) by one new feature – the first principal component – keeping as much variance as possible.



Step 1: Calculate the Mean

$$\bar{x}_j = \frac{1}{n}\sum_{i=1}^{n} x_{ij}$$

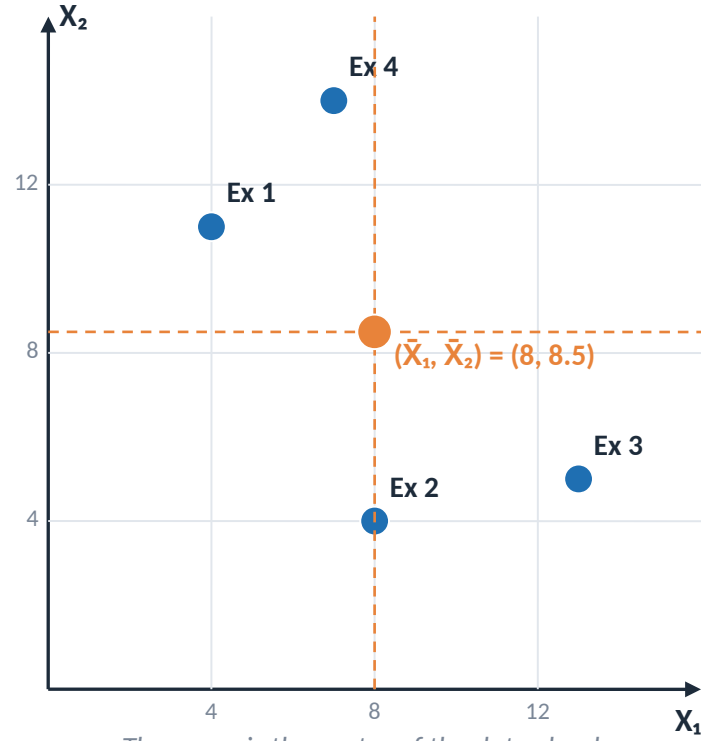
$$\bar{X}_1 = \frac{1}{4}(4+8+13+7) = 8$$

$$\bar{X}_2 = \frac{1}{4}(11+4+5+14) = 8.5$$

Centred data (each value minus its mean) – used in Steps 2 and 5:

	Ex 1	Ex 2	Ex 3	Ex 4
$X_1 - 8$	−4.0	0.0	5.0	−1.0
$X_2 - 8.5$	2.5	−4.5	−3.5	5.5

Each centred row sums to 0



The mean is the centre of the data cloud.

Step 2: Calculate the Covariance Matrix

$$S = \begin{bmatrix} \text{Cov}(X_1, X_1) & \text{Cov}(X_1, X_2) \\ \text{Cov}(X_2, X_1) & \text{Cov}(X_2, X_2) \end{bmatrix} = \begin{bmatrix} 14 & -11 \\ -11 & 23 \end{bmatrix}$$

Diagonal = variances.
Off-diagonal = covariances.
N – Number of examples

$$\text{Cov}(X_1, X_1) = \frac{1}{N-1} \sum_{k=1}^{N} (X_{1k} - \bar{X}_1)(X_{1k} - \bar{X}_1) = \frac{1}{3}((4-8)^2 + (8-8)^2 + (13-8)^2 + (7-8)^2) = 14$$

$$\text{Cov}(X_1, X_2) = \frac{1}{3}((4-8)(11-8.5) + (8-8)(4-8.5) + (13-8)(5-8.5) + (7-8)(14-8.5)) = -11$$

$$\text{Cov}(X_2, X_1) = \text{Cov}(X_1, X_2) = -11$$

$$\text{Cov}(X_2, X_2) = \frac{1}{3}((11-8.5)^2 + (4-8.5)^2 + (5-8.5)^2 + (14-8.5)^2) = 23$$

What S tells us: X_2 varies more than X_1 (23 > 14), and the covariance −11 is negative: when X_1 goes up, X_2 tends to go down.

Step 3: Eigenvalues of the Covariance Matrix

The characteristic equation of the covariance matrix is:

$$\det(S - \lambda I) = 0$$

$$\det\begin{bmatrix} 14 - \lambda & -11 \\ -11 & 23 - \lambda \end{bmatrix}$$

$$(14 - \lambda)(23 - \lambda) - (-11)^2 = 0$$

$$\lambda^2 - 37\lambda + 201 = 0$$

$$\lambda = \frac{1}{2}\left(37 \pm \sqrt{565}\right) = 30.3849,\ \ 6.6151$$

Eigenvalues

$\lambda_1 = 30.3849$

$\lambda_2 = 6.6151$

Meaning: each eigenvalue is the variance of the data along its eigenvector.

Step 4: Computation of the Eigenvectors

Let the eigenvector be U = (u_1, u_2). It must satisfy $(S-\lambda I)\,\mathbf{u}=0$

$$\begin{bmatrix} 14-\lambda & -11 \\ -11 & 23-\lambda \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

This gives two equations:

$$(14-\lambda)\,u_1 - 11\,u_2 = 0 \qquad -11\,u_1 + (23-\lambda)\,u_2 = 0$$

From the first equation, $(14-\lambda)u_1 = 11u_2$, so:

$$\frac{u_1}{11} = \frac{u_2}{14-\lambda} = t \;\Rightarrow\; u_1 = 11t,\;\; u_2 = (14-\lambda)t$$

Taking t = 1:

$$U = \begin{bmatrix} 11 \\ 14-\lambda \end{bmatrix}$$

- The two equations are not independent (det = 0), so one equation is enough.
- Any multiple t of U is also an eigenvector – only the direction matters.
- Next: put λ_1 and λ_2 into U and scale to length 1.

Step 4: Unit Eigenvectors e_1 and e_2

To find a unit eigenvector, divide U by its length ‖U‖.

For $\lambda_1 = 30.3849$:

$$\|U_1\| = \sqrt{11^2 + (14 - \lambda_1)^2} = \sqrt{11^2 + (14 - 30.3849)^2} = 19.7348$$

$$e_1 = \left(\frac{11}{19.7348}, \frac{14 - 30.3849}{19.7348}\right) = (0.5574,\ -0.8303)$$

For $\lambda_2 = 6.6151$:

$$\|U_2\| = \sqrt{11^2 + (14 - 6.6151)^2} = 13.2490$$

$$e_2 = \left(\frac{11}{13.2490}, \frac{14 - 6.6151}{13.2490}\right) = (0.8303,\ 0.5574)$$

e_1 belongs to the largest eigenvalue, so it is the first principal component direction.

Eigenvector matrix

$$E = \begin{bmatrix} 0.5574 & 0.8303 \\ -0.8303 & 0.5574 \end{bmatrix}$$

e_1 e_2

columns are e_1 and e_2

Checks: $e_1 \cdot e_2 = 0$ (perpendicular).
$S\,e_1 = (16.94, -25.23) = 30.3849 \times e_1$ ✓

Which Component Do We Keep?

Component	Eigenvalue (variance)
PC1 (e_1)	**30.3849**
PC2 (e_2)	6.6151

- Sort eigenvectors by eigenvalue, largest first.
- To go from 2 to 1 dimension we keep only e_1:

Step 5: First Principal Components

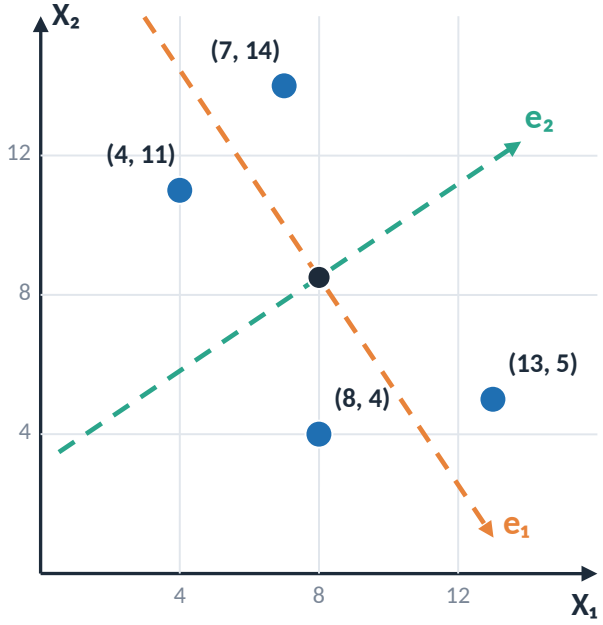
$$PC_1 = e_1^T(X_k - \bar{X}) = 0.5574\,(X_{1k} - 8) - 0.8303\,(X_{2k} - 8.5)$$

$$\text{Ex 1}: \quad 0.5574\,(4 - 8) - 0.8303\,(11 - 8.5) = -2.2296 - 2.0758 = -4.3054$$

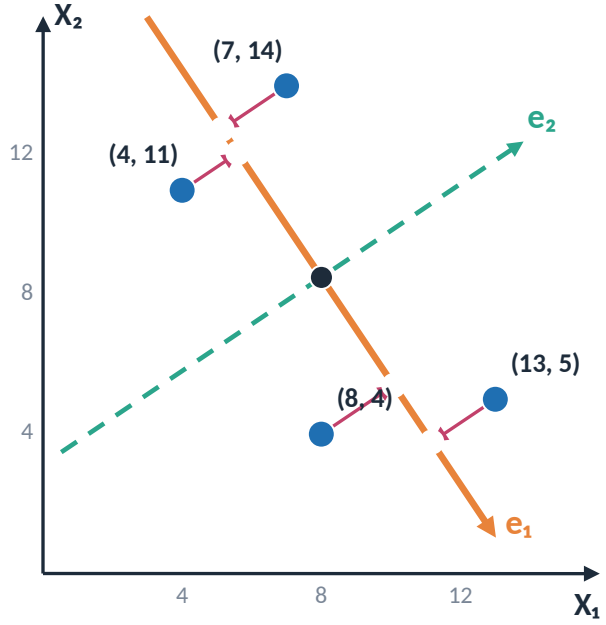
Example	Substitution	PC1
Ex 1	0.5574(4 − 8) − 0.8303(11 − 8.5)	**−4.3052**
Ex 2	0.5574(8 − 8) − 0.8303(4 − 8.5)	**3.7361**
Ex 3	0.5574(13 − 8) − 0.8303(5 − 8.5)	**5.6928**
Ex 4	0.5574(7 − 8) − 0.8303(14 − 8.5)	**−5.1238**

Feature	Ex 1	Ex 2	Ex 3	Ex 4
X_1	4	8	13	7
X_2	11	4	5	14
First principal component	**−4.3052**	**3.7361**	**5.6928**	**−5.1238**

Step 6: Geometrical Meaning of PC1



New axes e_1, e_2 through the mean (8, 8.5)



Each point is projected onto e_1

- e_1 and e_2 are the new, perpendicular axes, centred at the mean.
- e_1 lies along the direction of greatest spread.
- The PC1 value of each point is its signed position along e_1.
- We keep only $e_1 \rightarrow$ 1 number per example
- project all datapoints to e1

Eigenvectors determine the directions onto which data is projected, and eigenvalues help us select the most important directions.

PCA selects the directions with maximum variance because they retain the greatest amount of variation in the data.

Advantages, Limitations and Uses

Advantages

- Reduces features → faster training, less memory
- Removes correlated (redundant) features
- Reduces noise and overfitting
- Makes high-dimensional data easy to visualise in 2-D or 3-D

Limitations

- New components are hard to interpret (mixtures of features)
- Only captures linear relationships
- Sensitive to feature scale – standardise first
- Some information is always lost

Applications

- Image compression and face recognition (eigenfaces)
- Visualising high-dimensional data
- Pre-processing before other ML models
- Genetics and finance data analysis

Remember: PCA keeps the directions of largest variance. That is usually where the information is, but not always – check that the result still works for your task.

Key Takeaways

1. Dimensionality reduction keeps the information, drops extra dimensions
2. PCA finds new perpendicular axes ordered by how much variance they capture
3. Mean → centre → covariance matrix S → eigenvalues $\det(S - \lambda I) = 0$
4. Eigenvectors $(S - \lambda I)u = 0$, normalised; eigenvalue = variance along it
5. Keep the top-k eigenvectors and project: $PC_1 = e_1^T(X - \bar{X})$ (here 82.1% kept)